

if they don't directly appear in your own code. For example, you may have a secure, well-written application, but if it depends on a library with a critical flaw—like the infamous Log4j vulnerability (Log4Shell) or Heartbleed—your entire system could be at risk. SCA helps by not only identifying the vulnerable component but also recommending safe upgrade paths or patched versions.

In a DevSecOps pipeline, SCA is typically run early in the continuous integration process and continuously monitored during production to catch new vulnerabilities as they are disclosed. Many tools, such as Snyk, WhiteSource (now Mend), Sonatype Nexus Lifecycle, and GitHub Dependabot, integrate seamlessly with version control systems and CI/CD pipelines to automatically flag and even fix insecure dependencies before they're deployed.

Beyond security, SCA also plays a compliance role by tracking the licences associated with open source components. This ensures that your project respects legal obligations—important for avoiding costly legal disputes or compliance violations, especially in commercial products.

In essence, SCA acts as the security scanner for your supply chain: it inspects every package, library, and dependency that enters your project, making sure you're not accidentally importing vulnerabilities or legal headaches along with your code. In a world where most of an application's code is sourced from third parties, SCA is not optional—it's a necessity for building trustworthy, maintainable, and secure software.

Container and runtime security

In the modern DevOps world, applications are increasingly packaged and deployed as containers—lightweight, portable units that bundle an application with everything it needs to run, from libraries to configurations.

Platforms like Docker and container orchestration systems such as Kubernetes have revolutionised how software is built, shipped, and scaled. But with this convenience comes a new set of security challenges. Container and runtime security focuses on protecting both the containerised application and the environment it runs in, from the moment an image is built to its execution in production.

The first layer of container security starts during the image build phase. Containers are often created from base images pulled from public registries like Docker Hub. If these images contain outdated packages or malicious code, they can become an entry point for attackers. Security best practices here include using minimal, trusted base images, keeping images updated, and scanning them with tools like Trivy, Clair, or Anchore to detect known vulnerabilities. Additionally, container images should be signed and verified to ensure integrity, preventing tampering between the build and deployment stages.

Once containers are running, runtime security comes into play. Even a secure image can be exploited if the application inside is misconfigured, over-privileged, or exposed to an attack. Runtime security involves continuously monitoring container behaviour to detect suspicious activities such as unexpected network connections, privilege escalations, cryptomining scripts, or file system changes. Tools like Falco, Sysdig Secure, and Aqua Security specialise in watching container workloads for anomalies in real time, triggering alerts or even blocking malicious actions before they escalate.

Kubernetes and other orchestration platforms add another layer of complexity—and risk—since they manage multiple containers across clusters. Misconfigurations in Kubernetes, such as exposed dashboards, overly permissive role-

based access control (RBAC) policies, or unencrypted communication, can lead to cluster-wide compromises. To address this, security policies should be enforced through tools like OPA Gatekeeper or Kyverno, ensuring that only compliant workloads are deployed. Network segmentation, secrets management, and strong authentication for the control plane are also key measures.

In a DevSecOps workflow, container and runtime security checks are woven into the pipeline: images are scanned during continuous integration, policies are enforced during deployment, and monitoring runs continuously in production. This 'build, ship, run' security model ensures threats are detected and mitigated at every stage, reducing the attack surface.

In essence, container and runtime security acts as a lock, alarm system, and security guard for your cloud-native workloads—locking the container's contents, watching its behaviour for suspicious activity, and intervening before threats cause damage. Without it, a single vulnerable image or misconfigured container could compromise not just an application, but the entire infrastructure it runs on.

Infrastructure as Code (IaC) security

Infrastructure as Code (IaC) is one of the most transformative practices in modern DevOps. Instead of manually setting up servers, networks, and storage, teams use code—written in tools like Terraform, Ansible, Pulumi, or AWS CloudFormation—to define and provision infrastructure in a repeatable, automated way. This approach dramatically speeds up deployments, improves consistency, and reduces human error. However, because infrastructure is now defined as code, it also means infrastructure misconfigurations can be deployed just as quickly as application bugs, making security a critical concern from day one.